



دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش کار با Git

پیاده‌سازی بازی وردل فارسی به صورت Static Frontend

آزمایشگاه مهندسی نرم‌افزار	درس
دکتر حسین آبادی	استاد
تابستان ۱۴۰۵	ترم
سروش پورنصیری — ۴۰۰۱۰۴۸۵۶ آرمین محمودی‌نژاد — ۴۰۱۱۷۰۶۴۸	دانشجویان
github.com/seenpeh/Wordle-Software-Engineering	مخزن
seenpeh.github.io/Wordle-Software-Engineering	آدرس سایت

۱. معرفی نرم افزار

بازی وردل فارسی یک نرم افزار **static frontend** است که با HTML، CSS و JavaScript خالص نوشته شده و هیچ فریم ورک، وابستگی زمان اجرا یا مرحله‌ی build ندارد. بازیکن باید یک کلمه‌ی پنج حرفی فارسی را در شش نوبت حدس بزند و پس از هر حدس، رنگ خانه‌ها وضعیت هر حرف را نشان می‌دهد: سبز یعنی حرف در جای درست است، زرد یعنی در کلمه هست ولی جایش اشتباه است و خاکستری یعنی در کلمه وجود ندارد.

ماژول‌ها از طریق **رویداد** با هم صحبت می‌کنند و نه با فراخوانی مستقیم. این تصمیم معماری باعث شد دو عضو تیم بتوانند هم‌زمان و بدون وابستگی به کد یکدیگر پیش بروند؛ کیبورد بدون آنکه از وجود هسته‌ی بازی خبر داشته باشد رویداد ورودی منتشر می‌کند و ماژول آمار صرفاً به رویداد پایان بازی گوش می‌دهد.

آمار

وردل فارسی

پ	ر	ن	د	ه
ب	ا	ز	ا	ر
س	ت	ا	ر	ه

آفرین! درست حدس زدی.

ض	ص	ث	ق	ف	غ	ع	ه	خ	ح	ج	چ
ش	س	ی	ب	ل	ا	ت	ن	م	ک	گ	
ثب	ژ	ظ	ط	ز	ر	ذ	د	پ	و		

تصویر ۱ — نمای بازی پس از سه حدس. سبز یعنی حرف در جای درست، زرد یعنی جابه‌جا و خاکستری یعنی در کلمه نیست.

index.html	ساختار صفحه، کیبورد مجازی و مودال آمار
css/styles.css	تم، چیدمان گرید و کیبورد، طراحی واکنش‌گرا
js/config.js	ثابت‌های بازی: طول کلمه، تعداد حدس، وضعیت حروف
js/board.js	ساخت گرید شش در پنج و به‌روزرسانی خانه‌ها
js/state.js	بافر حدس در حال تایپ و وضعیت بازی
js/dictionary.js	دیکشنری خودکار ساخته: ۱۰,۱۶۷ کلمه‌ی معتبر و ۸۶۸ کلمه‌ی هدف
js/words.js	واسط دسترسی به دیکشنری و انتخاب کلمه‌ی هدف
js/evaluate.js	مقایسه‌ی حدس با کلمه‌ی هدف
js/game.js	کنترلر بازی: اتصال ورودی به وضعیت، رنگ‌آمیزی، پایان بازی
js/input.js	نرمال‌سازی ورودی، شامل تبدیل «ي» و «ك» عربی
js/keyboard.js	کیبورد مجازی و فیزیکی و بازخورد رنگی کلیدها
js/stats.js	ذخیره‌سازی آمار در localStorage
js/stats-modal.js	نمایش آمار در dialog با اعداد فارسی
tests/	۲۸ آزمون با node --test

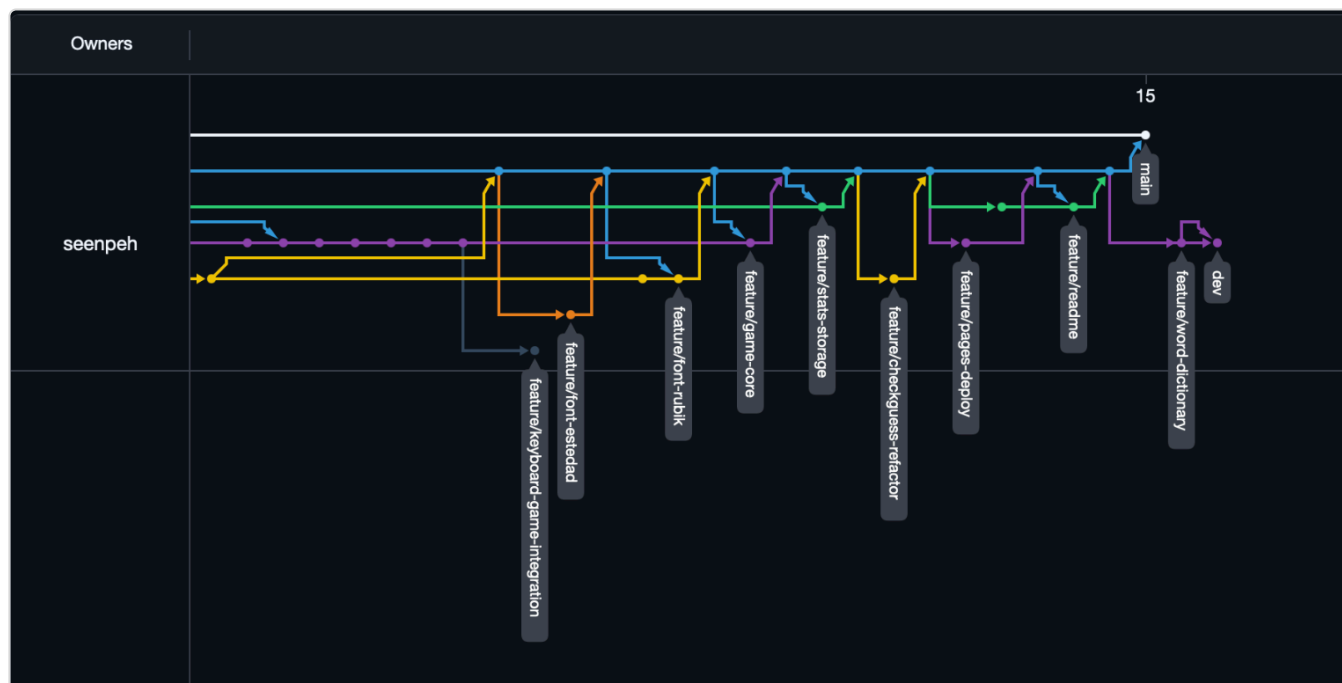
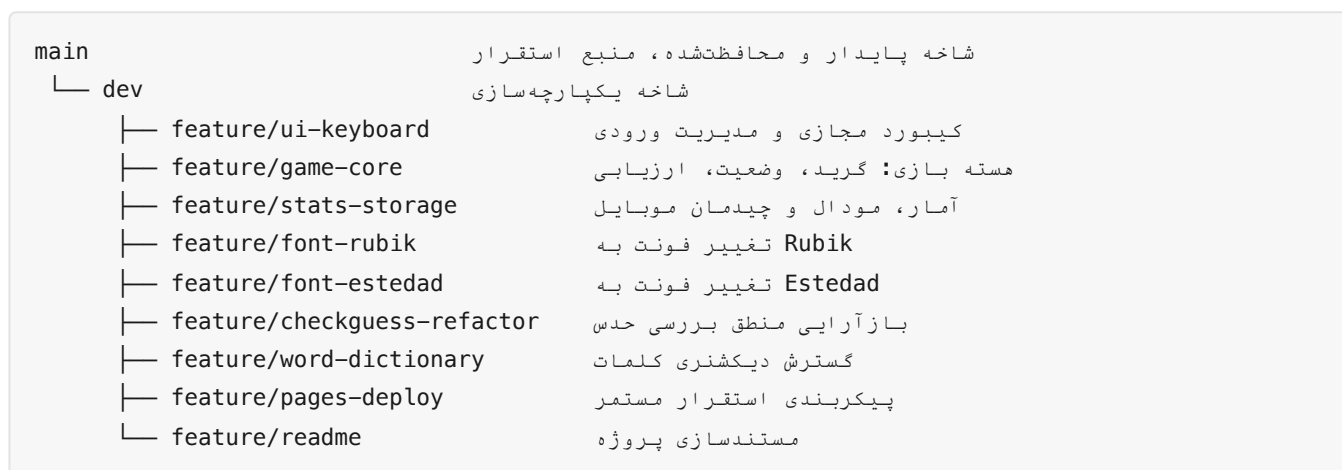
۲. تحقق اهداف آزمایش

هدف	وضعیت	شرح
استفاده از gitignore.	انجام شد	در نخستین کامیت پروژه اضافه شد و .env، dist/، node_modules/، DS_Store و فایل‌های ویرایشگر را کنار می‌گذارد.
حداقل ۲۰ کامیت معنادار	۲۴ کامیت	بدون احتساب ۱۶ کامیت merge. هر کامیت یک اتفاق مشخص را ثبت می‌کند و از قرارداد Conventional Commits پیروی می‌کند.
حداقل سه شاخه‌ی معنادار	۱۲ شاخه	سه نوع شاخه با اهداف متفاوت: main پایدار، dev برای یکپارچه‌سازی و نُه شاخه‌ی feature/*.
حداقل دو تعارض حل شده	سه تعارض	یکی از آن‌ها عمداً برنامه‌ریزی شد تا سناریوی تغییر هم‌زمان یک خط توسط دو نفر تمرین شود.
محافظت از شاخه‌ی main	فعال	ادغام فقط از راه pull request، با الزام حداقل یک تأیید. push مستقیم، force push و حذف شاخه مسدود است.
ادغام از راه pull request	۱۲ درخواست	هیچ شاخه‌ای به‌صورت مستقیم ادغام نشده است.

استقرار مستمر با GitHub Actions	فعال	هر push روی main ابتدا آزمون‌ها را اجرا و سپس سایت را منتشر می‌کند.
گزارش در README	انجام شد	شامل معماری، شاخه‌ها، کامیت‌ها، تعارض‌ها و آدرس صفحه‌ی GitHub Pages.

۳. ساختار شاخه‌ها

سه نوع شاخه با اهداف متفاوت به کار رفت. شاخه‌ی main همیشه قابل استقرار می‌ماند و هیچ‌کس مستقیماً روی آن کامیت نمی‌کند. شاخه‌ی dev محل یکپارچه‌سازی است و تعارض‌ها آنجا حل می‌شوند، پیش از آنکه چیزی به main برسد. هر شاخه‌ی feature/* یک قابلیت مستقل را در بر می‌گیرد.



تصویر ۲ — نمودار شبکه‌ی مخزن. هر شاخه‌ی feature از dev جدا می‌شود و پس از تکمیل به آن بازمی‌گردد. خط سفید بالا شاخه‌ی main است که فقط در نقاط انتشار به‌روز می‌شود.

۴. فرآیند کامیت‌ها

کار میان دو عضو تیم تقسیم شد: هسته‌ی بازی بر عهده‌ی فروش پورنصیری و رابط کاربری بر عهده‌ی آرمین محمودی‌نژاد. جدول زیر بیست کامیت نخست را نشان می‌دهد؛ چهار کامیت بعدی مربوط به گسترش دیکشنری، پیکربندی استقرار و مستندسازی است.

#	شناسه	نویسنده	اتفاق
۱	7336179	سروش	راه‌اندازی مخزن، gitignore و اسکلت اولیه
۲	019016c	آرمین	چیدمان کیبورد مجازی فارسی
۳	996cb4a	آرمین	اتصال کیبورد مجازی به کنش‌های بازی
۴	e84062b	آرمین	پشتیبانی از کیبورد فیزیکی
۵	3cbbbc7	آرمین	رنگ‌آمیزی کلیدها بر اساس بازخورد حدس
۶	31ff82b	آرمین	ذخیره نتایج بازی در localStorage
۷	ca4fe50	آرمین	نمایش نتیجه در مودال دسترس‌پذیر
۸	741ab8e	آرمین	بهینه‌سازی چیدمان برای موبایل
۹	df8b806	سروش	ساخت ساختار پوشه‌ها و توکن‌های تم
۱۰	303bc9f	سروش	رندر گرید شش ردیفی در پنج حرفی
۱۱	2080701	آرمین	تعریف Rubik به‌عنوان فونت اصلی
۱۲	129f19c	سروش	تغییر فونت پایه به Estedad
۱۳	ab58ff4	سروش	مدیریت حدس در حال تایپ
۱۴	18f2c6f	سروش	افزودن دیکشنری کلمات پنج‌حرفی
۱۵	b9e8ab3	سروش	ارزیابی حدس در برابر کلمه هدف
۱۶	be0c91b	سروش	رنگ‌آمیزی خانه‌ها بر اساس نتیجه
۱۷	a3e876c	سروش	مدیریت برد و باخت و پیام نتیجه
۱۸	a5023e2	آرمین	استفاده از Estedad به‌عنوان فونت اصلی
۱۹	83bd237	آرمین	تعریف متغیر فونت Rubik
۲۰	b52f43a	آرمین	بازآرایی منطق بررسی حدس در checkGuess

۵. تعارض‌ها و نحوه‌ی حل آن‌ها

در هر سه تعارض، کار هیچ‌یک از دو عضو تیم دور ریخته نشد و همواره هر دو طرف تلفیق شدند.

تعارض یکم — ادغام `dev` در `feature/game-core`

هر دو عضو تیم مستقلاً فایل‌های پای‌های یکسانی ساخته بودند، بنابراین Git هیچ جد مشترکی برای مقایسه نداشت و سه تعارض هم‌زمان از نوع `add/add` و `content` رخ داد. در `index.html` اسکلت معنایی `dev` مبنا قرار گرفت و گرید بازی جای `placeholder` نشست. در `app.js` هر دو راه‌اندازی ترکیب شدند. در `styles.css` هر دو استایل‌شیت نگه داشته شد و رنگ‌های هاردکدشده‌ی کیبورد به متغیرهای مشترک تبدیل شدند.

تعارض دوم — تعارض عمدی روی فونت

این تعارض به‌صورت برنامه‌ریزی‌شده ایجاد شد. یک شاخه فونت اصلی را `Estedad` کرد و شاخه‌ی دیگر همان خط را به `Rubik` تغییر داد. هنگام ادغام درخواست دوم، GitHub تعارض را گزارش کرد. به‌جای انتخاب یک‌طرفه، هر دو تعریف فونت نگه داشته شد و ترتیب فونت‌ها به‌صورت `Tahoma`, `Arial`, `"Rubik"`, `"Estedad"` حل شد؛ `Estedad` برای متن فارسی و `Rubik` به‌عنوان جایگزین برای متون لاتین.

تعارض سوم — ادغام مجدد `dev` در `feature/game-core`

پس از حل تعارض فونت، شاخه‌ی هسته‌ی بازی دوباره ادغام شد و `styles.css` تعارض گرفت. هنگام حل این تعارض یک اشکال واقعی هم کشف شد: آدرس `CDN` فونت `Estedad` خطای ۴۰۴ می‌داد، زیرا `jsDelivr` برچسب `latest@` را برای مخازن `gh` پشتیبانی نمی‌کند. آدرس به نسخه‌ی پین‌شده اصلاح شد و بارگذاری صحیح هر دو فونت در مرورگر تأیید شد.

۶. درخواست‌های ادغام و استقرار

تمام دوازده ادغام پروژه از راه `pull request` انجام شد. شاخه‌ی `main` با قانون محافظت، ادغام مستقیم را رد می‌کند و حداقل یک تأیید لازم دارد. این محدودیت در عمل هم آزموده شد: تلاش برای ادغام درخواست دوازدهم با پیام `base branch policy prohibits the merge` رد شد.

☐ 1 Open ☒ 11 Closed	Author ▾	Label ▾	Projects ▾	Milestones ▾	Reviews ▾	Assignee ▾	Sort ▾
☐ گسترش دیکشنری کلمات فارسی #11 by seenpeh (Owner) was merged 29 minutes ago							
☐ انتشار نسخه اول وردل فارسی #10 by seenpeh (Owner) was merged 1 hour ago · Review required							
☐ افزودن مستندات و گزارش کامل پروژه #9 by seenpeh (Owner) was merged 15 hours ago							
☐ GitHub Pages پیکربندی استقرار مستمر روی #8 by seenpeh (Owner) was merged 15 hours ago							
☐ checkGuess بازآرایی منطق بررسی حدس در تابع #7 by seenpeh (Owner) was merged 15 hours ago							
☐ افزودن آمار بازی، مودال نتیجه و چیدمان موبایل #6 by seenpeh (Owner) was merged 15 hours ago							
☐ ادغام هسته بازی با رابط کاربری و کیبورد #5 by seenpeh (Owner) was merged 15 hours ago							
☐ Rubik تغییر فونت اصلی به #4 by arminmahmudinezhad-gif (Collaborator) was merged 15 hours ago							
☐ Estedad تغییر فونت اصلی به #3 by arminmahmudinezhad-gif (Collaborator) was merged 15 hours ago							
☐ Rubik تغییر فونت اصلی به #2 by arminmahmudinezhad-gif (Collaborator) was merged 16 hours ago							
☐ افزودن صفحه کلید فارسی و مدیریت ورودی بازی #1 by arminmahmudinezhad-gif (Collaborator) was merged 17 hours ago							

تصویر ۳ — فهرست درخواست‌های ادغام. برچسب نویسنده نشان می‌دهد که کار میان دو عضو تیم تقسیم شده است.

Branch name pattern ^{*}

main

Applies to 1 branch

main

Protect matching branches

☒ **Require a pull request before merging**
 When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

☒ **Require approvals**
 When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

Required number of approvals before merging: 1 ▾

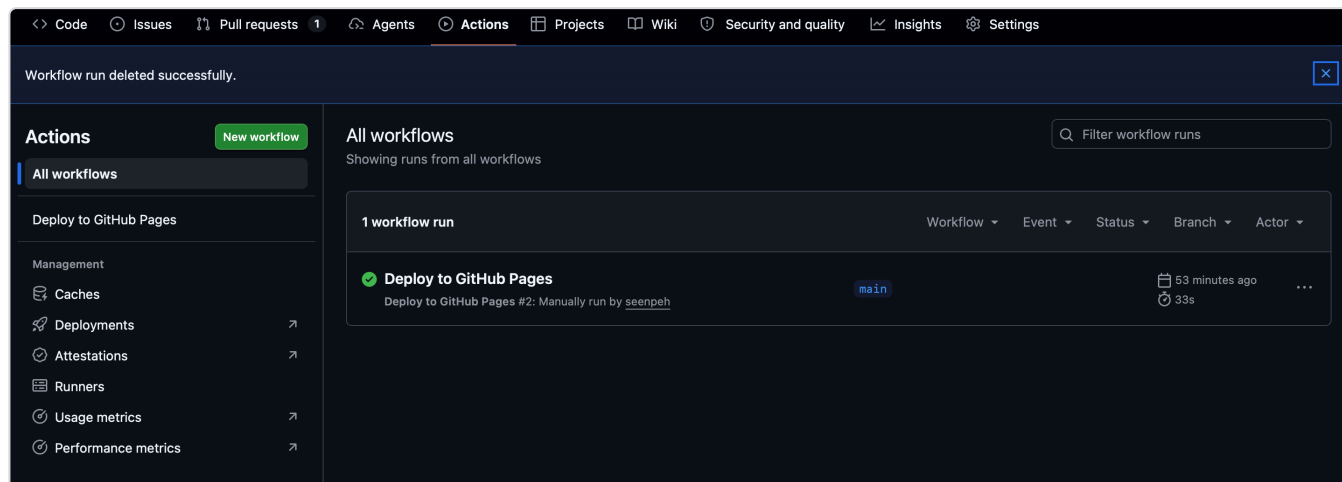
☐ **Dismiss stale pull request approvals when new commits are pushed**
 New reviewable commits pushed to a matching branch will dismiss pull request review approvals.

☐ **Require review from Code Owners**
 Require an approved review in pull requests including files with a designated code owner.

☐ **Require approval of the most recent reviewable push**
 Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

تصویر ۴ — قانون محافظت شاخه‌ی main: الزام درخواست ادغام و حداقل یک تأیید.

استقرار با GitHub Actions انجام می‌شود. گردش کار نخست آزمون‌ها را اجرا می‌کند و مرحله‌ی انتشار با `needs: test` به آن وابسته است؛ پس اگر آزمون‌ی شکست بخورد هیچ نسخه‌ی معیوبی منتشر نمی‌شود. چون پروژه `static` است و مرحله‌ی `build` ندارد، کل مخزن مستقیماً منتشر می‌گردد.



تصویر ۵ — اجرای موفق گردش کار Deploy to GitHub Pages روی شاخه‌ی main.

وضعیت آزمون‌ها: ۲۸ آزمون با `node --test` اجرا می‌شود و همگی موفق هستند. این آزمون‌ها نرمال‌سازی ورودی، اولویت وضعیت کلیدها، اعتبار دیکشنری، ارزیابی حدس و محاسبات آمار را پوشش می‌دهند.

۷. پاسخ به پرسش‌ها

پاسخ‌ها بر پایه‌ی استاندارد `ASD-STE100` نوشته شده‌اند.

۱. پوشه‌ی git چیست؟

پوشه‌ی `git`. پایگاه داده‌ی مخزن است. این پوشه تمام تاریخچه‌ی پروژه را نگه می‌دارد. پوشه این داده‌ها را ذخیره می‌کند: اشیای `Git` شامل `commit` و `tree` و `blob`، شاخه‌ها و برچسب‌ها، ناحیه‌ی `stage`، پی‌کربندی مخزن، مخازن راه دور و گزارش حرکت‌های `HEAD`. دستور `git init` این پوشه را می‌سازد. دستور `git clone` نیز آن را از یک مخزن راه دور کپی می‌کند.

۲. منظور از atomic بودن چیست؟

یک `commit` اتمی فقط یک تغییر منطقی را در بر می‌گیرد. تمام فایل‌های آن به یک هدف مربوط هستند. اگر شما آن `commit` را برگردانید، هیچ کار دیگری خراب نمی‌شود.

یک `pull request` اتمی نیز فقط یک قابلیت یا یک اصلاح را ارائه می‌کند. بازبین می‌تواند آن را ساده بفهمد. تیم می‌تواند آن را مستقل از کارهای دیگر بپذیرد یا رد کند.

۳. تفاوت `fetch` و `pull` و `merge` و `rebase` و `cherry-pick`

تغییرات مخزن راه دور را دریافت می‌کند. این دستور شاخه‌ی کاری شما را تغییر نمی‌دهد.	<code>fetch</code>
ابتدا <code>fetch</code> را اجرا می‌کند و سپس تغییرات را در شاخه‌ی کاری ادغام می‌کند. این دستور برابر است با <code>fetch</code> به همراه <code>merge</code> .	<code>pull</code>

merge	دو شاخه را با یک commit جدید به هم می‌پیوندند. تاریخچه‌ی اصلی دست‌نخورده می‌ماند.
rebase	commit‌های شما را روی نوک شاخه‌ی دیگر دوباره اعمال می‌کند. نتیجه یک تاریخچه‌ی خطی است، اما شناسه‌ی commit‌ها تغییر می‌کند.
cherry-pick	فقط یک commit مشخص را از شاخه‌ای دیگر روی شاخه‌ی جاری اعمال می‌کند.

در این پروژه از cherry-pick استفاده شد. یک شاخه‌ی هم‌تیمی `commit` تکراری را با خود حمل می‌کرد. تیم فقط همان یک `commit` بازآرایی را برداشت و تاریخچه‌ی `dev` تمیز ماند.

۴. تفاوت `checkout` و `switch` و `restore` و `revert` و `reset`

reset	نشانگر شاخه را به یک commit پیشین می‌برد. این دستور تاریخچه را بازنویسی می‌کند. آن را روی شاخه‌ی مشترک اجرا نکنید.
revert	یک commit جدید می‌سازد که اثر commit پیشین را خنثی می‌کند. تاریخچه حفظ می‌شود. این روش برای شاخه‌ی مشترک ایمن است.
restore	محتوای فایل‌ها را از commit یا از ناحیه‌ی stage بازمی‌گرداند.
switch	شاخه‌ی جاری را عوض می‌کند. این دستور فقط همین یک کار را انجام می‌دهد.
checkout	دستور قدیمی است و هر دو کار بالا را انجام می‌دهد. Git دستورهای <code>switch</code> و <code>restore</code> را جایگزین آن کرد، زیرا <code>checkout</code> بیش از حد چندکاره بود.

۵. ناحیه‌ی stage و دستور `stash`

ناحیه‌ی stage یا index یک ناحیه‌ی میانی میان پوشه‌ی کاری و مخزن است. شما تغییرات را با `git add` به این ناحیه می‌برید. سپس `git commit` فقط همین تغییرات را ثبت می‌کند. این ناحیه به شما اجازه می‌دهد که یک commit اتمی بسازید.

دستور `git stash` تغییرات ناتمام را موقتاً کنار می‌گذارد و پوشه‌ی کاری را پاک می‌کند. شما می‌توانید شاخه را عوض کنید. سپس با `git stash pop` تغییرات را بازمی‌گردانید.

۶. مفهوم `snapshot` و ارتباط آن با `commit`

هر commit یک snapshot کامل از تمام فایل‌های پروژه در یک لحظه است. commit یک مجموعه تفاوت یا diff ذخیره نمی‌کند.

Git برای فایل‌های تغییرنکرده شیء تازه نمی‌سازد و به همان شیء پیشین اشاره می‌کند. پس فضای ذخیره‌سازی هدر نمی‌رود. آن diff که شما در `git show` می‌بینید، در لحظه‌ی نمایش و از مقایسه‌ی دو snapshot ساخته می‌شود. این طراحی، دستورهای مانند `checkout` و `merge` را سریع می‌کند، زیرا Git تفاوت‌ها را دوباره محاسبه نمی‌کند.

۷. تفاوت مخزن محلی و مخزن راه دور

مخزن محلی روی رایانه‌ی شما قرار دارد. شما بدون اتصال به شبکه در آن کار می‌کنید. تمام تاریخچه در دسترس شماست.

مخزن راه دور روی یک سرور مانند GitHub قرار دارد. این مخزن نقطه‌ی اشتراک تیم است. شما با push کار خود را به آن می‌فرستید و با fetch یا pull کار دیگران را دریافت می‌کنید. قابلیت‌هایی مانند pull request و قانون محافظت شاخه فقط روی مخزن راه دور وجود دارند.